

Laporan Tugas Besar 1
IF3270 PEMBELAJARAN MESIN
FEED-FORWARD NEURAL NETWORK
SEMESTER II TAHUN 2025/2026



Muhammad Izzat Jundy	13523092
Zulfaqqar Nayaka Athadiansyah	13523094
Muhammad Adha Ridwan	13523098

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

Daftar Isi

1	Deskripsi Persoalan	2
2	Analisis dan Pembahasan	2
2.1	Implementasi	2
2.2	Penjelasan Forward Propagation	3
3	Hasil Pengujian	5
3.1	CNN	5
3.2	Simple RNN dan LSTM	14
4	Penutup	14
4.1	Kesimpulan	14
4.2	Saran	14
4.3	Pembagian Tugas	14
	Referensi	15
A	Surat Pernyataan Penggunaan AI	16

1 Deskripsi Persoalan

2 Analisis dan Pembahasan

2.1 Implementasi

2.1.1 Modul CNN Layers

Berikut merupakan kelas-kelas layer yang diimplementasikan from scratch pada modul CNN.

Table 1: Atribut dan Metode Kelas-kelas CNN

Kelas	Atribut / Metode	Deskripsi
Conv2D	weight, bias stride, pad _forward(x) forward(x)	Parameter bobot konvolusi dan bias. Langkah dan padding operasi konvolusi. Operasi feedforward pada satu buah citra. Operasi konvolusi batch / channel support.
LocallyConnected2D	weight, bias, ... forward(x)	Bobot tidak dibagi ukurannya sesuai kernel/patch. Forward propagation operasi locally connected.
Dense	weights, bias forward(x)	Bobot linear (fully connected layer). Feedforward ($X \cdot W + b$) dan aktivasi.
Flatten	forward(x) _forward(x)	Mengubah tensor multidimensi menjadi vektor 1D. Implementasi <code>reshape(-1)</code> .
MaxPooling2D	pool_size, stride forward(x)	Ukuran dan stride pooling filter. Melakukan max pooling window spatial.
AveragePooling2D	pool_size, stride forward(x)	Ukuran dan stride pooling filter. Melakukan average pooling window spatial.
GlobalAveragePooling2D	forward(x)	Mean pooling di seluruh dimensi spatial.
GlobalMaxPooling2D	forward(x)	Max pooling di seluruh dimensi spatial.

2.1.2 Modul RNN & LSTM Layers

Berikut merupakan kelas-kelas layer sekuensial yang diimplementasikan from scratch pada modul RNN dan LSTM.

Table 2: Atribut dan Metode Kelas-kelas RNN/LSTM

Kelas	Atribut / Metode	Deskripsi
SimpleRNNCell	<code>kernel</code>	Bobot transformasi input pada setiap timestep.
	<code>recurrent_kernel</code> <code>forward(x.t, h_prev)</code>	Bobot rekursif untuk <i>hidden state</i> RNN. Perhitungan satu step state RNN.
SimpleRNN	<code>cell</code> <code>forward(x, init_st)</code>	Menggunakan SimpleRNNCell. Unroll / loop t sequence waktu untuk RNN.
	<code>kernel, bias</code> <code>recurrent_kernel</code> <code>forward(x, h, c)</code>	Bobot gabungan (4 gates) dan bias. Bobot berulang untuk state sebelumnya. Step kalkulasi input gate, forget gate, kandidat memori, dan output gate.
LSTMCell	<code>kernel, bias</code> <code>recurrent_kernel</code> <code>forward(x, h, c)</code>	Bobot gabungan (4 gates) dan bias. Bobot berulang untuk state sebelumnya. Step kalkulasi input gate, forget gate, kandidat memori, dan output gate.
	<code>cell</code> <code>forward(x, init_st)</code>	Menggunakan LSTMCell. Unroll step-by-step memory tracking.
LSTM	<code>weights</code> <code>forward(token_ids)</code>	Vector vocabulary size $\times$ dimension mapping. Konversi integer token ids menjadi vektor dense.
	<code>from_keras(layer)</code>	Load transfer matriks embedding Keras.

2.2 Penjelasan Forward Propagation

2.2.1 CNN

Forward propagation pada CNN menerima masukan citra berbentuk tensor $X \in \mathbb{R}^{N \times H \times W \times C}$, dengan N sebagai jumlah sampel dalam batch, H dan W sebagai ukuran spasial citra, dan C sebagai jumlah kanal. Pada implementasi Conv2D, input tunggal berdimensi (H, W, C) terlebih dahulu diubah menjadi batch berukuran satu, sedangkan input batch diproses langsung. Apabila parameter `pad` lebih besar dari nol, citra diberi padding nol pada sisi spasial sebelum operasi konvolusi dilakukan.

Untuk setiap posisi keluaran (i, j) dan filter ke- k , layer konvolusi mengambil patch lokal berukuran $k_H \times k_W \times C$ dari input. Nilai keluaran dihitung dengan menjumlahkan hasil perkalian elemen patch dengan bobot filter, lalu menambahkan bias:

$$Y_{n,i,j,k} = \sum_{u=0}^{k_H-1} \sum_{v=0}^{k_W-1} \sum_{c=0}^{C-1} X_{n,i+s+u,j+s+v,c} W_{u,v,c,k} + b_k$$

dengan s sebagai stride. Ukuran keluaran spasial dihitung sebagai:

$$H_{out} = \left\lfloor \frac{H_{pad} - k_H}{s} \right\rfloor + 1, \quad W_{out} = \left\lfloor \frac{W_{pad} - k_W}{s} \right\rfloor + 1.$$

Pada kode, patch input dibentuk menggunakan `np.lib.stride_tricks.as_strided` agar seluruh patch dapat dipandang sebagai tensor enam dimensi. Patch tersebut kemudian diubah menjadi matriks kolom, bobot filter diubah menjadi matriks W_{col} , dan keluaran dihitung melalui perkalian matriks `col @ W_col + bias`. Jika layer memiliki fungsi aktivasi, misalnya ReLU, aktivasi diterapkan setelah operasi linear tersebut.

Pada `LocallyConnected2D`, alurnya mirip dengan `Conv2D`, tetapi bobot tidak dibagi antarposisi spasial. Artinya, setiap lokasi keluaran memiliki parameter sendiri. Jika jumlah lokasi keluaran adalah $L = H_{out} W_{out}$, maka bobot memiliki indeks lokasi sehingga perhitungan dapat ditulis sebagai:

$$Y_{n,l,o} = \sum_i P_{n,l,i} W_{l,i,o} + b_{l,o}.$$

Implementasi menggunakan `np.einsum` untuk menghitung perkalian patch dengan bobot lokal pada setiap lokasi. Pendekatan ini membuat *locally connected layer* lebih fleksibel dibanding konvolusi biasa, tetapi jumlah parameternya jauh lebih besar karena tidak ada *weight sharing*.

Setelah konvolusi, pooling digunakan untuk mereduksi ukuran spasial feature map. Pada **MaxPooling2D**, setiap window pooling menghasilkan nilai maksimum:

$$Y_{n,i,j,c} = \max_{(u,v) \in \Omega_{i,j}} X_{n,u,v,c}.$$

Pada **AveragePooling2D**, nilai keluaran adalah rata-rata elemen dalam window:

$$Y_{n,i,j,c} = \frac{1}{|\Omega_{i,j}|} \sum_{(u,v) \in \Omega_{i,j}} X_{n,u,v,c}.$$

Layer **Flatten** kemudian mengubah tensor feature map menjadi vektor satu dimensi agar dapat masuk ke layer **Dense**. Pada layer **Dense**, forward propagation dihitung sebagai:

$$Y = XW + b,$$

diikuti fungsi aktivasi apabila tersedia. Dengan demikian, alur CNN secara umum adalah ekstraksi fitur lokal melalui konvolusi, reduksi spasial melalui pooling, perubahan bentuk melalui flatten, dan klasifikasi atau proyeksi akhir melalui dense layer.

2.2.2 RNN

Forward propagation pada Simple RNN digunakan untuk memproses data sekuensial, misalnya urutan embedding token caption. Input RNN direpresentasikan sebagai $X \in \mathbb{R}^{N \times T \times D}$, dengan N sebagai batch size, T sebagai panjang sekuens, dan D sebagai dimensi fitur pada setiap timestep. Jika input hanya berbentuk (T, D) , implementasi menambahkan dimensi batch sementara agar proses komputasi tetap seragam.

Simple RNN menyimpan informasi masa lalu dalam hidden state h_t . Pada timestep ke- t , **SimpleRNNCell** menerima input x_t dan hidden state sebelumnya h_{t-1} . State baru dihitung dengan:

$$h_t = \phi(x_t W_x + h_{t-1} W_h + b),$$

dengan W_x sebagai **kernel**, W_h sebagai **recurrent_kernel**, b sebagai bias, dan ϕ sebagai fungsi aktivasi, yaitu **tanh** pada implementasi ini. Jika **initial_state** tidak diberikan, hidden state awal diinisialisasi sebagai vektor nol:

$$h_0 = \mathbf{0}.$$

Layer **SimpleRNN** melakukan *unrolling* sepanjang timestep. Untuk setiap indeks waktu, layer memanggil `cell.forward(x[:, t, :], h_t)` dan menyimpan hasilnya ke daftar keluaran. Jika **go_backwards=True**, urutan dibaca dari timestep terakhir menuju timestep pertama, kemudian hasilnya dikembalikan ke urutan waktu semula. Jika **return_sequences=True**, seluruh hidden state $[h_1, h_2, \dots, h_T]$ dikembalikan sebagai tensor (N, T, U) . Jika **return_sequences=False**, hanya hidden state terakhir yang dikembalikan sebagai representasi sekuens.

Pada pipeline captioning, token caption terlebih dahulu diproses oleh **Embedding**. Forward propagation embedding hanya melakukan indexing pada matriks embedding:

$$E_t = W_{emb}[\text{token}_t].$$

Fitur citra dari encoder CNN diproyeksikan oleh layer dense, lalu disisipkan sebagai timestep awal decoder. Dengan teacher forcing, input decoder adalah gabungan fitur citra terproyeksi dan embedding token ground-truth sebelumnya. Output RNN kemudian diproyeksikan oleh dense layer akhir untuk menghasilkan logit atau probabilitas vocabulary pada setiap timestep.

2.2.3 LSTM

LSTM digunakan untuk mengatasi keterbatasan Simple RNN dalam menyimpan konteks jangka panjang. Berbeda dari Simple RNN yang hanya membawa hidden state, LSTM membawa dua state: hidden state h_t dan cell state c_t . Hidden state berperan sebagai keluaran jangka pendek, sedangkan cell state menyimpan memori yang dapat diteruskan lebih stabil sepanjang sekuens.

Pada setiap timestep, **LSTMCell** menghitung satu transformasi linear besar untuk empat komponen gate sekaligus:

$$z = x_t W_x + h_{t-1} W_h + b.$$

Vektor z kemudian dibagi menjadi empat bagian:

$$z_i, z_f, z_c, z_o = \text{split}(z).$$

Masing-masing bagian digunakan untuk menghitung input gate, forget gate, kandidat memori, dan output gate:

$$i_t = \sigma(z_i), \quad f_t = \sigma(z_f), \quad \tilde{c}_t = \tanh(z_c), \quad o_t = \sigma(z_o).$$

Cell state baru dihitung dengan menggabungkan memori lama dan kandidat memori baru:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t.$$

Forget gate f_t menentukan bagian memori lama yang dipertahankan, sedangkan input gate i_t menentukan bagian kandidat memori baru yang dimasukkan. Hidden state baru dihitung dari cell state yang sudah diperbarui:

$$h_t = o_t \odot \tanh(c_t).$$

Output gate o_t mengontrol seberapa besar isi cell state yang diekspos sebagai keluaran timestep tersebut.

Implementasi LSTM melakukan unroll dengan pola yang sama seperti SimpleRNN, tetapi setiap timestep memperbarui dua state sekaligus: `h_t` dan `c_t`. Jika `initial_state` tidak diberikan, kedua state diinisialisasi dengan nol. Layer juga mendukung `return_sequences`, `return_state`, dan `go_backwards` agar perilakunya konsisten dengan layer Keras. Pada decoder captioning, keluaran LSTM pada setiap timestep diteruskan ke dense projection untuk menghasilkan distribusi probabilitas token berikutnya. Pada saat inferensi greedy, token dengan probabilitas terbesar dipilih, ditambahkan ke prefix, lalu proses forward diulang sampai token `<end>` muncul atau panjang maksimum caption tercapai.

3 Hasil Pengujian

Bagian ini menyajikan hasil eksperimen model CNN serta RNN/LSTM yang dijalankan melalui Jupyter Notebook. Pengujian mencakup variasi arsitektur, parameter pelatihan, hingga perbandingan performa dengan pustaka standar Keras sebagai *benchmark*.

3.1 CNN

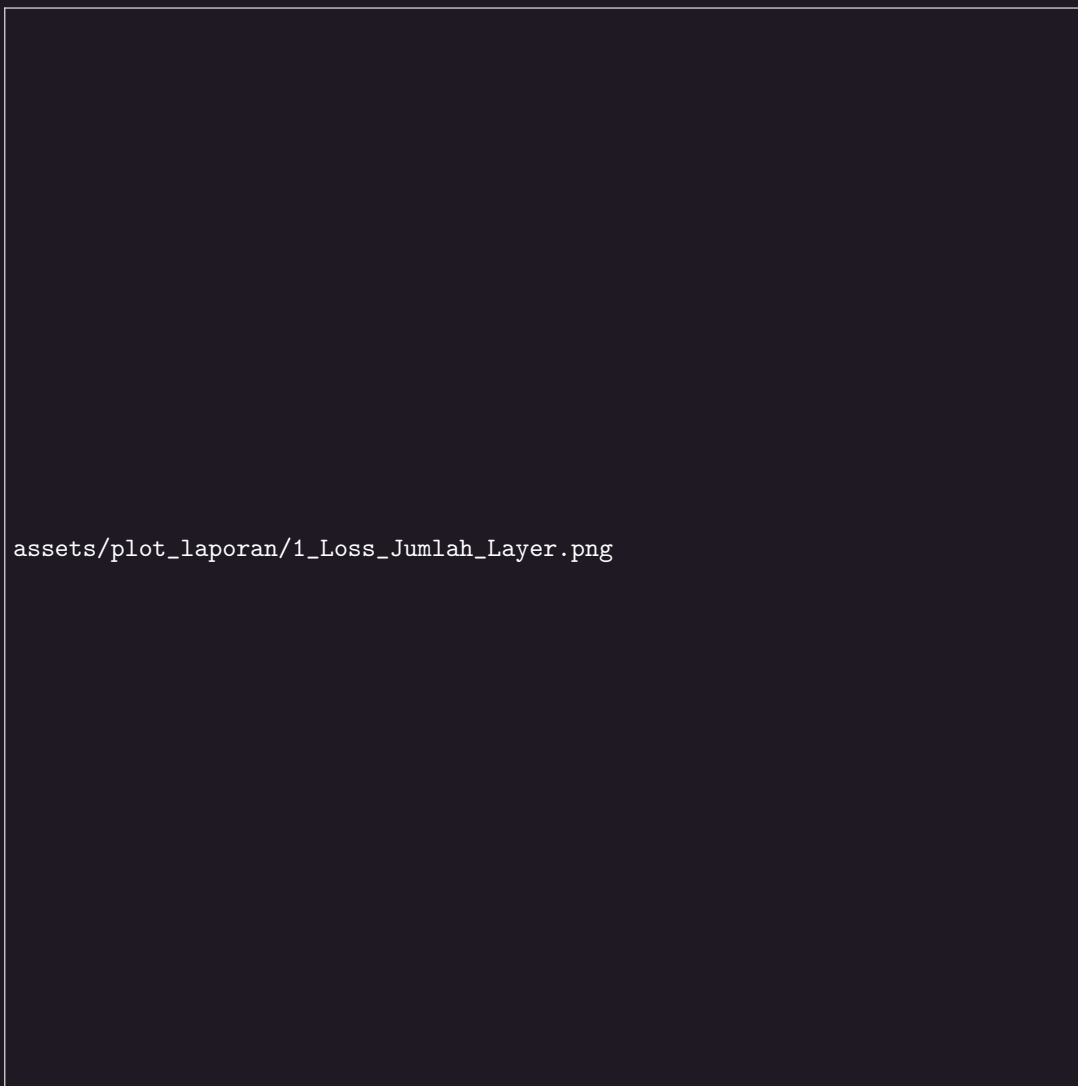
Berdasarkan hasil eksperimen pencarian parameter (*hyperparameter sweep*) 16 konfigurasi menggunakan pustaka Keras, diperoleh analisis dari berbagai komponen variasi arsitektur CNN sebagai berikut. Parameter performa utama yang diukur adalah F1-Score (*Macro*) pada himpunan data uji (*test set*) dan waktu pelatihan model.

3.1.1 Pengaruh Jumlah Layer Konvolusi

Peningkatan kedalaman arsitektur dari 2 layer menjadi 3 layer secara signifikan meningkatkan performa model (F1-Macro naik dari 0.7607 menjadi 0.8058). Hal ini menunjukkan bahwa arsitektur yang lebih dalam mampu mengekstraksi representasi fitur hierarkis yang lebih kompleks. Namun, penambahan layer ini juga mengakibatkan peningkatan beban komputasi pelatihan, dari rata-rata 110.07 detik menjadi 181.56 detik.

assets/plot_laporan/1_F1_Jumlah_Layer.png

Gambar 1: Rata-rata F1-Score berdasarkan Jumlah Layer



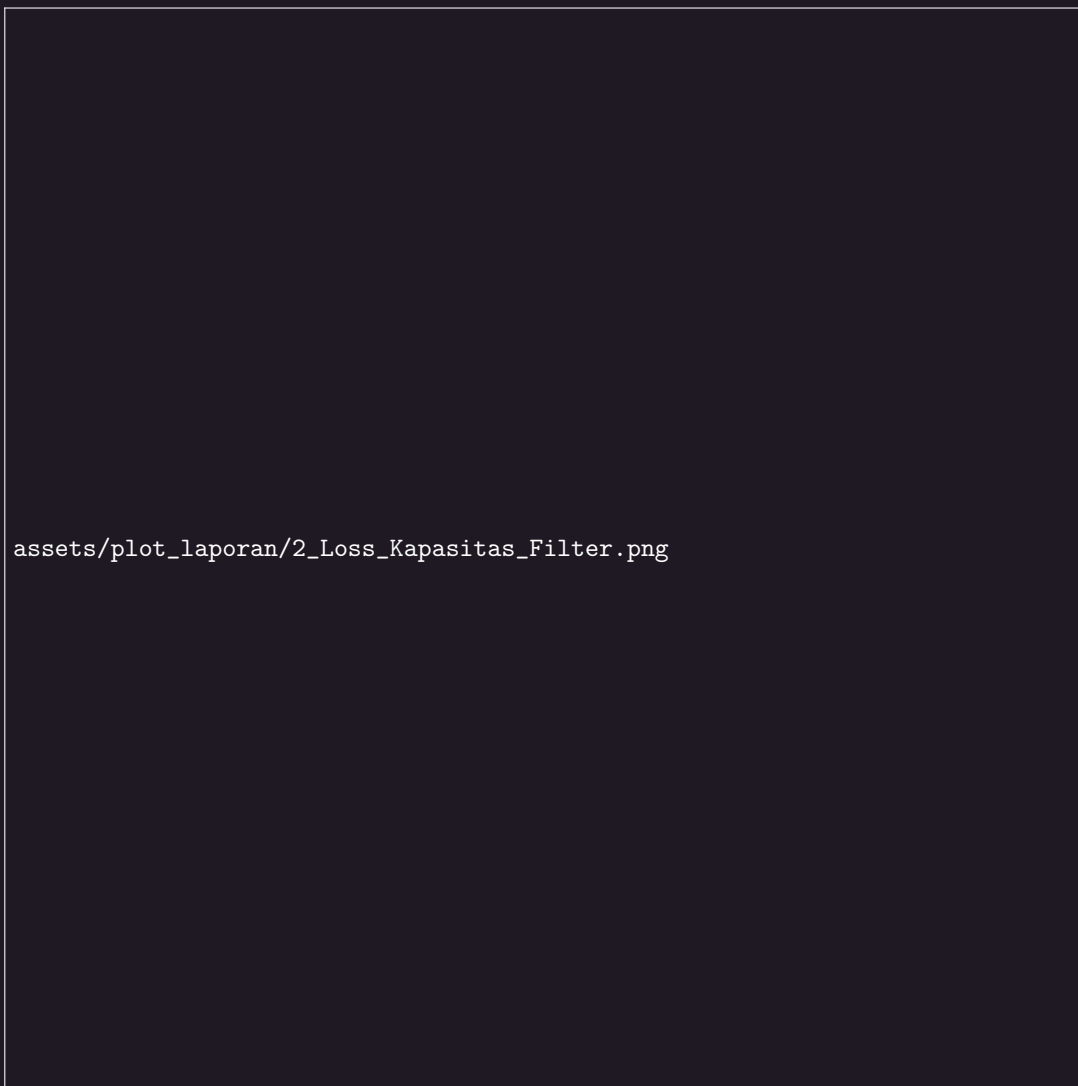
Gambar 2: Grafik Loss Model Terbaik: 2 Layer vs 3 Layer

3.1.2 Pengaruh Kapasitas Model (Banyak Filter)

Arsitektur dengan kapasitas parameter filter besar (dimulai dari 64 filter) sedikit mengungguli arsitektur filter kecil (dimulai dari 32 filter) dengan perolehan skor F1-Macro 0.7843 berbanding 0.7822. Perbedaan kinerja klasifikasi tersebut tidak terlalu eskalatif jika meninjau waktu latih yang harus dibayar. Model berkapasitas besar memakan waktu komputasi nyaris dua kali lipat lebih lama (191.80 detik) dibandingkan model yang lebih ringan (99.83 detik).

assets/plot_laporan/2_F1_Kapasitas_Filter.png

Gambar 3: Rata-rata F1-Score berdasarkan Kapasitas Filter



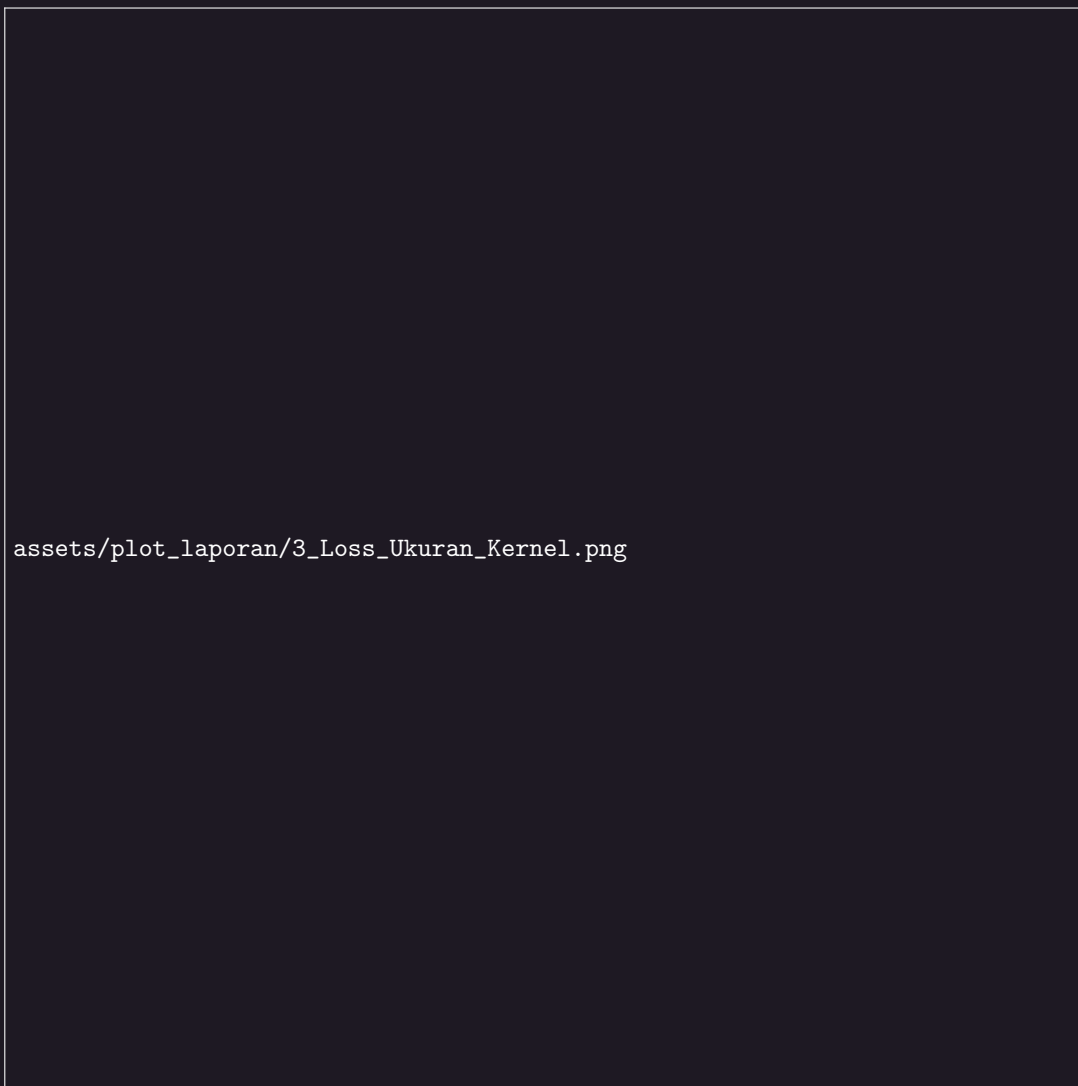
Gambar 4: Grafik Loss Model Terbaik: Filter Kecil vs Besar

3.1.3 Pengaruh Ukuran Kernel

Eksperimen membuktikan bahwa penggunaan resolusi kernel konvolusi 3×3 memberikan performa yang jauh lebih baik (F1-Macro 0.7912) daripada agregasi informasi lebih besar yang dilakukan oleh ukuran 5×5 (F1-Macro 0.7753). Kernel 3×3 lebih unggul menangkap detail spasial. Model dengan kernel 3×3 juga lebih hemat waktu komputasi latih (117.63 detik berbanding 174.00 detik).

assets/plot_laporan/3_F1_Ukuran_Kernel.png

Gambar 5: Rata-rata F1-Score berdasarkan Ukuran Kernel



Gambar 6: Grafik Loss Model Terbaik: Ukuran Kernel 3×3 vs 5×5

3.1.4 Pengaruh Jenis Pooling Layer

Model klasifikasi yang menerapkan *Max Pooling* secara rata-rata lebih superior (F1-Macro 0.7913) bila dibandingkan dengan penerapan *Average Pooling* (0.7752). Hal ini dikarenakan operasi max meretensi nilai intensitas terkuat dari kumpulan fitur ketimbang meratakannya secara menyeluruh. *Max pooling* berjalan dengan rerata waktu latih 157.54 detik.

assets/plot_laporan/4_F1_Jenis_Pooling.png

Gambar 7: Rata-rata F1-Score berdasarkan Jenis Pooling Layer

assets/plot_laporan/4_Loss_Jenis_Pooling.png

Gambar 8: Grafik Loss Model Terbaik: Max Pooling vs Average Pooling

3.1.5 Perbandingan Keras vs *From Scratch*

3.1.6 Perbandingan Arsitektur *Shared* vs *Non-shared*

3.2 Simple RNN dan LSTM

3.2.1 Perbandingan RNN dan LSTM

3.2.2 Perbandingan dengan Keras

3.2.3 Pengaruh Panjang Maksimum Caption terhadap BLEU-4

4 Penutup

4.1 Kesimpulan

4.2 Saran

4.3 Pembagian Tugas

Table 3: Pembagian tugas anggota kelompok

NIM	Nama	Tugas
13523092	Muhammad Izzat Jundy	lorem ipsum
13523094	Zulfaqqar Nayaka Athadiansyah	lorem ipsum
13523098	Muhammad Adha Ridwan	lorem ipsum

Referensi

1. Karpathy, A. *The Spelled-Out Intro to Neural Networks and Backpropagation: Building Micrograd*. <https://youtu.be/VMj-3S1tku0>
2. Osajima, J. *Forward Propagation Explained*. <https://www.jasonosajima.com/forwardprop>
3. Osajima, J. *Backward Propagation Explained*. <https://www.jasonosajima.com/backprop>
4. NumPy Documentation. <https://numpy.org/doc/2.2/>
5. scikit-learn. *MLPClassifier*. https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html
6. OpenStax Calculus. *The Chain Rule for Multivariable Functions*. [https://math.libretexts.org/Bookshelves/Calculus/Calculus_\(OpenStax\)/14%3ADifferentiation_of_Functions_of_Several_Variables/14.05%3A_The_Chain_Rule_for_Multivariate_Functions](https://math.libretexts.org/Bookshelves/Calculus/Calculus_(OpenStax)/14%3ADifferentiation_of_Functions_of_Several_Variables/14.05%3A_The_Chain_Rule_for_Multivariate_Functions)
7. Green, E. *The Softmax Function and Its Derivative*. <https://eli.thegreenplace.net/2016/the-softmax-function-and-its-derivative/>
8. Orr, D. *Building Autodiff from Scratch*. <https://douglasorr.github.io/2021-11-autodiff/article.html>
9. Grosse, R. *CSC2541 Tutorial: Automatic Differentiation*. https://www.cs.toronto.edu/~rgrosse/courses/csc2541_2022/tutorials/tut01.pdf
10. Built In. *L2 Regularization*. <https://builtin.com/data-science/l2-regularization>

A Surat Pernyataan Penggunaan AI

Dengan ini, kami:

Nama/NIM : Zulfaqqar Nayaka Athadiansyah / 13523094
Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika
Kode Mata Kuliah : IF3270
Nama Mata Kuliah : Pembelajaran Mesin
Judul Tugas/Proposal : Tugas Besar 2: Convolutional Neural Network dan Recurrent Neural Network

menyatakan bahwa kami **menggunakan** AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika **Ya**, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: Grammarly, Paperpal, DeepL, Quillbot, ChatGPT, Gemini, dsb.	Tidak	
Pembuatan Teks: ChatGPT, Gemini, Copilot, Jasper, Jenni AI, dsb.	Tidak	
Bantuan Diskusi/Konten: Perplexity, ChatGPT, Zoom-Companion, dsb.	Ya	Untuk brainstorming awal, pembagian tugas, dan strategi implementasi
Grafik/Multimedia: DALL-E, Midjourney, Stable Diffusion, Canva AI, dsb.	Tidak	
Lainnya:	Tidak	

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini secara jujur dan transparan.

Bandung, 15 Mei 2026



Zulfaqqar Nayaka Athadiansyah